Phase 3: Logical Design and Normalization

Sports Club Management System — CS323 Database Systems

David Acheampong Awuah, Richard Yemoh, Samira Donkoh, Ronald Ocloo

---

1. Introduction

In this phase we converted the Phase 2 entity-relationship model into a relational schema and normalized it to Third Normal Form. The result is 12 tables, 12 foreign keys and 8 planned indexes. Every attribute has a data type, a key designation and a constraint, so the Phase 4 DDL can be written directly from this document. The revised model is shown in Figure 1.

2. Changes made to the Phase 2 model

No.	Change	Reason
C1	ATHLETE.name split into first_name and last_name; contact split into email and phone	First Normal Form requires atomic values. A single name column also makes FR8 unreliable, since we cannot sort or match on surname.
C2	MEMBERSHIP.type extracted into a new MEMBERSHIP_TYPE table	Third Normal Form. The fee depends on the type, not on the membership itself. Explained in Section 4.
C3	MEMBERSHIP.amount_charged added	Records the price paid at purchase. Not a copy of the current fee: without it, a later price change would rewrite past revenue figures.
C4	The two many-to-many relationships resolved into TEAM_ROSTER and TEAM_COMPETITION	The relational model cannot hold many-to-many directly. Flagged in Phase 2, Section 2.3, and implemented here.
C5	SPORT lookup table added	NFR5 asks for new sports without redesign. A lookup table needs one INSERT; an ENUM would need an ALTER TABLE, and free text invites spelling variants that break reports.
C6	APP_USER table added	FR10 (role-based access) and NFR2 (hashed passwords) had nowhere to live. Nothing in the Phase 2 model stored a password or a role.

3. Weak entities and their parent entities

A weak entity has no key of its own. It is identified only through its parent entities and cannot exist without them. In Figure 1 weak entities are drawn as double rectangles, and the relationships that identify them are drawn as double diamonds. Our model contains two.

Weak entity	Parent entities	Identifying relationships	Identifier
TEAM_ROSTER	ATHLETE, TEAM	Enrolled_In (to ATHLETE), Rosters (to TEAM)	athlete_id and team_id together, both borrowed from the parents
TEAM_COMPETITION	TEAM, COMPETITION	Registers (to TEAM), Receives (to COMPETITION)	team_id and competition_id together, both borrowed from the parents

Both weak entities have total participation in their identifying relationships, shown by the double lines on the diagram, and theirs are the only foreign keys in the schema set to CASCADE on delete. MEMBERSHIP, PAYMENT and FACILITY_BOOKING each have a mandatory foreign key, but they remain strong entities because each has its own surrogate primary key. Existence dependency alone does not make an entity weak; the test is whether it can be identified without its parent.

4. The normalization problem we found

Once a fee is added, the Phase 2 MEMBERSHIP relation carries the following functional dependencies:

membership_id → athlete_id, type, start_date, end_date, status
type → fee

Because fee is determined by type, and type is not a key, we have membership_id → type → fee. This is a transitive dependency, so the relation is in Second Normal Form but not Third. It produces an update anomaly, since raising the Junior fee means editing every Junior membership row; an insertion anomaly, since a new membership type cannot be recorded until somebody buys one; and a deletion anomaly, since removing the last Premium membership erases the fact that Premium costs 800.

We decomposed the relation into MEMBERSHIP_TYPE (type_id, type_name, fee, duration_months) and MEMBERSHIP, which now holds type_id as a foreign key. The decomposition is lossless, since joining on type_id rebuilds the original relation, and it is dependency-preserving. All 12 tables are now in Third Normal Form.

5. Design decisions

Facility bookings use fixed time slots with a UNIQUE constraint on facility_id, booking_date and time_slot, which turns Business Rule 5 from something a trigger has to catch into something the database cannot allow. Delete behaviour is RESTRICT on financial and historical links and CASCADE only on the two weak entities, and venue is stored as free text so that away fixtures can be recorded.

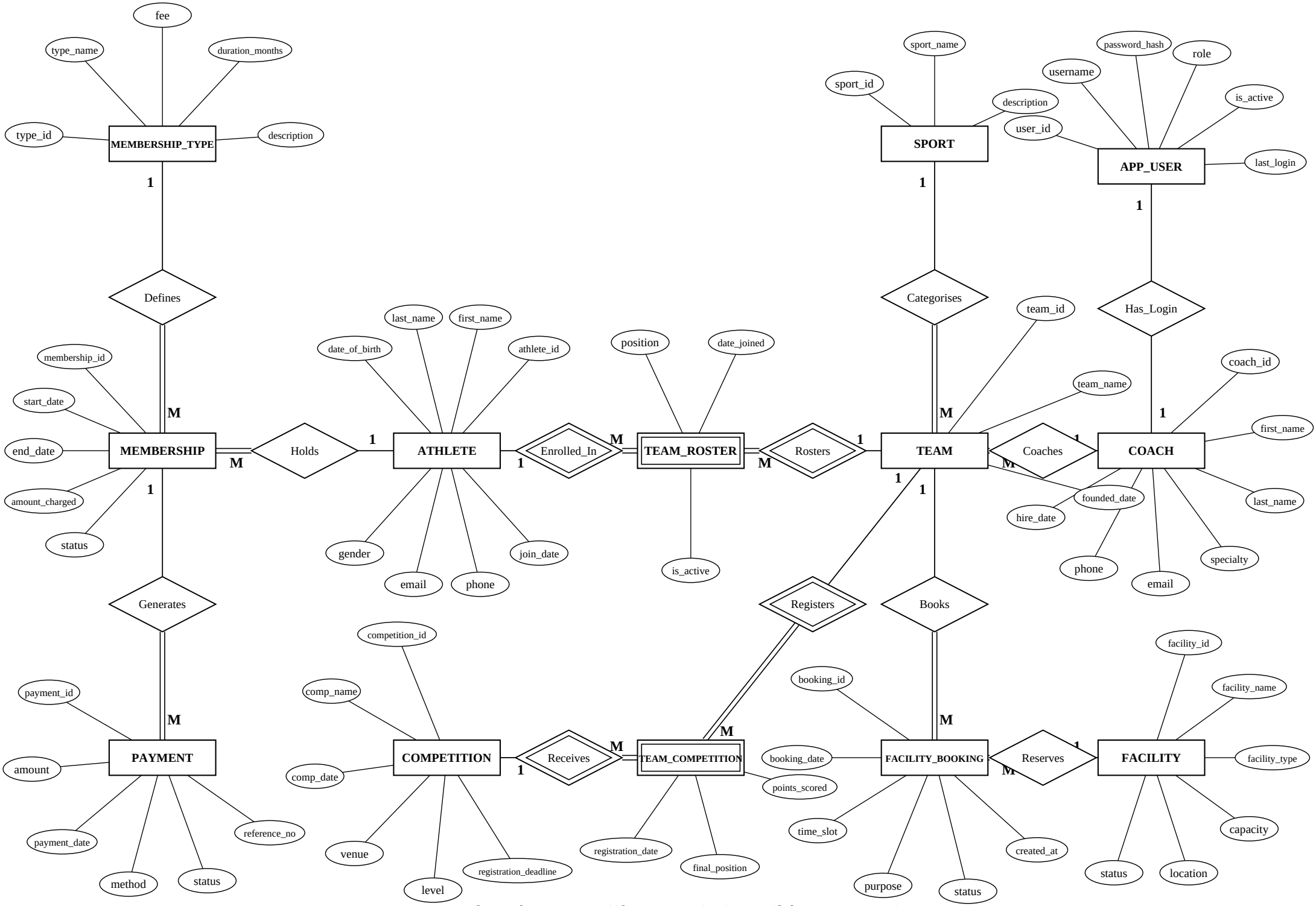

Figure 1. Entity-Relationship Diagram (Chen notation) - Sports Club Management System

6. Requirements met by the Phase 3 schema

Each requirement below is tied to the table, constraint or index that satisfies it. Items marked as later phases are supported by the schema but implemented by another member of the team.

6.1 Business rules

No.	Rule	How the schema meets it
BR1	An athlete must hold a membership before joining a team	Not expressible as a foreign key, being a conditional rule across two tables. Enforced by a BEFORE INSERT trigger on TEAM_ROSTER in Phase 6b.
BR2	Membership status is always Active, Expired or Suspended	MEMBERSHIP.status declared as an ENUM of those three values and NOT NULL. Any other value is rejected by the DBMS.
BR3	A payment belongs to exactly one membership	PAYMENT.membership_id as a NOT NULL foreign key. The NOT NULL is what prevents orphan payments.
BR4	A team has exactly one coach; a coach may lead many teams	A single NOT NULL foreign key, TEAM.coach_id. A nullable column would allow uncoached teams; a junction table would allow several coaches.
BR5	A facility cannot be double-booked for the same date and slot	A UNIQUE constraint on facility_id, booking_date and time_slot in FACILITY_BOOKING. The second booking is refused with no application code involved.
BR6	Teams enter many competitions and competitions include many teams	The TEAM_COMPETITION weak entity, keyed on team_id and competition_id together.
BR7	Only staff and admin may change payment and membership status	APP_USER.role for the application layer, together with GRANT privileges at database level in Phase 7.

6.2 Functional requirements

No.	Requirement	Supported by
FR1	Create, update and delete athlete records	ATHLETE, with deletion restricted where membership history exists
FR2	Create and renew memberships	MEMBERSHIP and MEMBERSHIP_TYPE. A renewal inserts a new row, so history is preserved
FR3	Record payments and track payment status	PAYMENT.amount, status, method and reference_no
FR4	Assign coaches to teams	TEAM.coach_id
FR5	Link athletes to teams	TEAM_ROSTER, carrying date_joined, position and is_active
FR6	Schedule competitions and assign teams to them	COMPETITION and TEAM_COMPETITION, which also stores the result
FR7	Book facilities and prevent conflicts	FACILITY_BOOKING with the UNIQUE slot constraint
FR8	Search and filter by name, status and date	Indexes on last_name, MEMBERSHIP.status and comp_date; names split for exact matching
FR9	Generate reports on members, revenue and bookings	MEMBERSHIP.status, amount_charged, PAYMENT.payment_date and booking_date
FR10	Enforce role-based access	APP_USER.role, with coach_id linking a coach login to their own teams

6.3 Non-functional requirements

No.	Requirement	How the schema addresses it
NFR1	Queries under two seconds	Eight planned indexes. The composite index on facility_id and booking_date matters most, since every booking attempt scans that pair for conflicts.
NFR2	Hashed passwords and restricted payment access	APP_USER.password_hash is sized at 255 characters, for bcrypt or Argon2 rather than the current algorithm. Payment access is limited by role.
NFR3	Backup and recovery	The database is rebuildable from three scripts in the schema folder and one in the data folder, with no manual setup step to be lost.
NFR4	Minimal training for front-desk staff	ENUM columns give the application fixed value lists for status, method, level and facility type, so those fields become dropdowns rather than free text.
NFR5	Growth to thousands of athletes and multiple sports	SPORT and MEMBERSHIP_TYPE are extended by INSERT rather than by schema change. Surrogate integer keys stay compact as row counts grow.
NFR6	Referential integrity at database level	Twelve foreign keys, each with an explicit ON DELETE rule. RESTRICT protects financial and historical data, and CASCADE applies only to the two weak entities.

Three of the seven business rules are enforced by structure alone, and BR5 by a single UNIQUE constraint. BR1 and BR7 require work in Phases 6b and 7, as they are conditional and privilege-based rules that no table definition can express.